

Training bot status console

Chapter 2 · Practical Lab 1 · Defining and using a simple class

In this lab, you will build a small C++ class from an empty Visual Studio project. The program models a training bot with health. The bot can take damage, report its current health, and tell the rest of the program whether it is still alive.

The aim is not to build a complicated game system. The aim is to practise the class basics from the lecture and guided tutorial until the file structure and syntax begin to feel familiar.

Duration: 120 minutes. This is a practice lab. You will create the solution, project and source files yourself; there is no starter project to open.

How to use this sheet. Work from top to bottom. Build and run whenever you see a **Checkpoint**. If your output does not match the example, fix that stage before moving on. You do not need to invent extra tests or write a test framework.

Before you begin

Use a Windows PC with **Visual Studio 2026**, the **Desktop development with C++** workload, Platform Toolset **v145** and a Windows SDK installed. This lab uses **C++17**, **/W4**, **Debug** and **x64**.

What you will practise

- creating a C++ Visual Studio project from scratch;
- creating a header file and source file for a class;
- distinguishing a class from an object;
- keeping data private and using public member functions;
- implementing a constructor and a member initialiser list;
- writing read-only member functions with `const`; and
- creating two objects from the same class and showing that each has its own state.

The problem

Imagine a simple training arena in a game. The arena can create training bots. Every bot stores one piece of information: its health. A new bot normally starts with 100 health. Damage reduces health, but health must never go below 0.

We will represent this with a class named `TrainingBot`. The class will keep its health private and provide small public operations that other code can use.

What the finished program will show

By the end of the core lab, your console should produce this output:

```
Training bot lab

First bot
Starting health: 100
After 25 damage: 75
Alive: true

Second bot
Starting health: 40
After 10 damage: 30
Alive: true

Heavy damage
Second bot health: 0
Second bot alive: false
```

You are not expected to memorise this output. It is here so that you always know what your program is working towards.

Your two-hour route

Stage	What you will do	Suggested time
01	Create the Visual Studio project and <code>Main.cpp</code> from scratch	20 min
02	Create <code>TrainingBot.h</code> and <code>TrainingBot.cpp</code>	20 min
03	Challenge 1 - make the bot take damage safely	20 min
04	Challenge 2 - add a read-only <code>isAlive()</code> question	15 min
05	Add a constructor for a chosen starting health	20 min
06	Challenge 3 - create two separate objects	15 min
07	Final check, troubleshooting and optional stretch	10 min

Build little and often. When a build fails, read the first useful error message, fix that problem, then build again. Do not keep typing several more stages on top of a broken program.

01 · Create the project from scratch

Start with an empty C++ project. The first goal is simply to make one line appear in the console. Do not create the class yet.

Step 1 - Open Visual Studio. Choose Create a new project.

Step 2 - Choose an Empty Project. Search for Empty Project, select the C++ Empty Project template, and continue.

Step 3 - Name the project. Use TrainingBotLab for the project name. Use the same name for the solution unless your lecturer tells you otherwise.

Step 4 - Check the build configuration. In the toolbar, select Debug and x64.

Step 5 - Set the language standard and warnings. Right-click the project in Solution Explorer and choose Properties. Apply the settings to All Configurations. Set Platform Toolset to v145, C/C++ → Language → C++ Language Standard to ISO C++17, and C/C++ → General → Warning Level to Level 4 (/W4). Apply the changes and close the dialog.

Step 6 - Create Main.cpp. Right-click Source Files → Add → New Item → C++ File (.cpp). Name it Main.cpp.

Type this into Main.cpp

```
#include <iostream>

int main()
{
    std::cout << "Training bot lab\n";
    return 0;
}
```

Step 7 - Build the solution. Use Build → Build Solution.

Step 8 - Run it. Use Debug → Start Without Debugging.

Expected output

```
Training bot lab
```

Checkpoint 1. Do not continue until the project builds and the heading appears. If Visual Studio reports an error, fix this small program first.

What you have done so far

You now have a working C++ console project. Everything that follows will be added to this same project. Keep exactly one main() function.

02 · Create the class files

Now we will create a very small class. The class will live in two files, just like the lecture and guided tutorial: the header describes the class, while the source file contains the member-function definitions.

File	Its job
TrainingBot.h	Declare the TrainingBot class and its public/private members.
TrainingBot.cpp	Define what the TrainingBot member functions do.
Main.cpp	Create TrainingBot objects and use their public functions.

Create TrainingBot.h

Step 1 - Add the header file. Right-click Header Files → Add → New Item → Header File (.h). Name it TrainingBot.h.

Type the complete header below. For now, the class has one private data member and one public read-only function.

```
#ifndef TRAINING_BOT_H
#define TRAINING_BOT_H

class TrainingBot
{
public:
    int health() const;

private:
    int m_health{100};
};

#endif
```

What this means. TrainingBot is the class name. m_health belongs to each object and starts at 100. private prevents Main.cpp from changing the value directly. health() const will let other code read the value.

Create TrainingBot.cpp

Step 2 - Add the source file. Right-click Source Files → Add → New Item → C++ File (.cpp). Name it TrainingBot.cpp.

Type this definition:

```
#include "TrainingBot.h"

int TrainingBot::health() const
{
    return m_health;
}
```

The TrainingBot:: prefix means that health() belongs to the TrainingBot class. The const suffix means this function reads the object without changing its ordinary member data.

Use the class in Main.cpp

Replace the current contents of Main.cpp with:

```
#include "TrainingBot.h"

#include <iostream>

int main()
{
    std::cout << "Training bot lab\n\n";

    TrainingBot bot{};
    std::cout << "Starting health: " << bot.health() << '\n';

    return 0;
}
```

Expected output

```
Training bot lab
```

```
Starting health: 100
```

Checkpoint 2. If the program prints 100, you have successfully defined a class, created an object and called one of its member functions.

03 · Challenge 1 - make the bot take damage

At the moment the bot can report its health, but nothing can change it. Your first challenge is to add a public operation named `takeDamage()`.

The rule

When the program calls `takeDamage(amount)`:

- a positive amount smaller than the current health is subtracted;
- an amount equal to or larger than the current health sets health to 0; and
- zero or a negative amount does nothing.

This keeps the bot health at 0 or above.

Step A - add the declaration

Open `TrainingBot.h`. In the `public:` section, add this line above `health()`:

```
void takeDamage(int t_amount);
```

Step B - add the definition

Open `TrainingBot.cpp`. Add this function below `health()`. Most of it is supplied. Complete the one missing statement.

```
void TrainingBot::takeDamage(int t_amount)
{
    if (t_amount <= 0)
    {
        return;
    }

    if (t_amount >= m_health)
    {
        m_health = 0;
    }
    else
    {
        // Challenge: subtract t_amount from m_health here.
    }
}
```

Hint. You already know subtraction. The shorter C++ form `m_health -= t_amount;` means the same as `m_health = m_health - t_amount;`.

Step C - call the function from Main.cpp

Keep your existing code. Immediately after the starting-health line, add:

```
bot.takeDamage(25);
std::cout << "After 25 damage: " << bot.health() << '\n';
```

Expected output

```
Training bot lab

Starting health: 100
After 25 damage: 75
```

Checkpoint 3. If the output says 75, your public function is changing the private state of this object through a controlled operation.

Quick boundary check

Change only the damage amount from 25 to 500 and run the program once. The result should be 0, not -400. Then change the amount back to 25 before continuing.

```
After 500 damage: 0
```

04 · Challenge 2 - ask whether the bot is alive

A useful class does not need to expose its private field. Instead, it can answer useful questions. We will add `isAlive()`, which returns true when health is greater than 0.

Step A - declare the function

Open `TrainingBot.h`. Add this line in the public section:

```
bool isAlive() const;
```

Step B - define the function

Open `TrainingBot.cpp`. Add the following skeleton and replace the comment with a Boolean expression.

```
bool TrainingBot::isAlive() const
{
    return /* true when m_health is greater than 0 */;
}
```

Hint. A comparison such as `m_health > 0` already produces either `true` or `false`.

Step C - print the answer

In `Main.cpp`, add `std::boolalpha` near the start of `main()` so Boolean values print as `true/false` instead of `1/0`. Then print `isAlive()`.

```
std::cout << std::boolalpha;

// Keep the rest of your code.
std::cout << "Alive: " << bot.isAlive() << '\n';
```

Expected output after 25 damage

```
Starting health: 100
After 25 damage: 75
Alive: true
```

Now check zero health

Temporarily add one more damage call after the 25 damage:

```
bot.takeDamage(1000);
std::cout << "After heavy damage: " << bot.health() << '\n';
std::cout << "Alive now: " << bot.isAlive() << '\n';
```

Expected additional output

```
After heavy damage: 0
Alive now: false
```

Why is `isAlive()` `const`? Because the function only observes the bot. It does not need writable access to `m_health`. The declaration and definition must both include the `const` suffix.

05 · Add a constructor for a chosen starting health

So far, every TrainingBot begins with 100 health because m_health has an in-class initial value. Sometimes we want to create a bot with a different valid starting value. This is what a constructor can provide.

For the core lab, use starting health values between 0 and 100. You are not being asked to validate constructor input here.

Step A - declare two construction routes

Open TrainingBot.h. Add these two declarations at the start of the public section:

```
TrainingBot() = default;  
TrainingBot(int t_health);
```

Read these as two choices. TrainingBot() creates the ordinary 100-health bot using the member default. TrainingBot(int) lets the caller supply a starting health.

Step B - define the one-argument constructor

Open TrainingBot.cpp. Put this definition below the include and before the other member functions:

```
TrainingBot::TrainingBot(int t_health)  
    : m_health{t_health}  
{  
}
```

The line after the colon is a member initialiser list. It gives m_health its starting value before the constructor body runs. This is the same constructor pattern introduced in the lecture and tutorial.

Step C - create a second bot

In Main.cpp, create another object with 40 health. Add these lines after your first-bot code:

```
std::cout << "\nSecond bot\n";  
TrainingBot secondBot{40};  
std::cout << "Starting health: " << secondBot.health() << '\n';
```

Expected new section

```
Second bot  
Starting health: 40
```

Checkpoint 4. If the second bot prints 40 while the first bot still started at 100, both constructor routes are working.

Common constructor mistake

Do not write `int TrainingBot::TrainingBot(...)`. Constructors do not have a return type. The function name is exactly the class name.

06 · Challenge 3 - show that two objects are separate

The lecture makes an important distinction: a class is one type, while each object created from that type has its own ordinary member data. Let us make that visible in the console.

Step A - give each bot different damage

Use two objects: one default bot and one 40-health bot. Apply different damage amounts:

```
TrainingBot firstBot{};
TrainingBot secondBot{40};

firstBot.takeDamage(25);
secondBot.takeDamage(10);
```

Step B - print both objects

Add output so that the first bot reports 75 and the second reports 30. You may use the supplied lines below instead of inventing your own formatting:

```
std::cout << "First bot health: " << firstBot.health() << '\n';
std::cout << "Second bot health: " << secondBot.health() << '\n';
```

Expected output

```
First bot health: 75
Second bot health: 30
```

What this proves. Both objects use the same TrainingBot class, but firstBot and secondBot store separate m_health values. Changing one object does not automatically change the other.

Step C - one small prediction

Before running, change secondBot.takeDamage(10); to secondBot.takeDamage(50);. Predict the two health values first. Then run the program.

Object	Expected health
firstBot	75 - it was not changed again.
secondBot	0 - damage cannot take health below zero.

After you have checked the result, restore the second-bot damage amount to 10 so that your final output matches the lab.

You have now used the core class ideas

- class definition;
- private member data;
- public member functions;
- an object created from a class;
- a default constructor and a custom constructor;
- a member initialiser list;
- a manipulator that changes state; and
- const accessors that read state.

07 · Put Main.cpp into its final form

At this point your class should already work. Replace only the contents of Main.cpp with the version below so that everyone finishes with the same simple demonstration. You are not adding a separate test system - the console output is the check for this lab.

```
#include "TrainingBot.h"

#include <iostream>

int main()
{
    std::cout << std::boolalpha;
    std::cout << "Training bot lab\n\n";

    TrainingBot firstBot{};

    std::cout << "First bot\n";
    std::cout << "Starting health: " << firstBot.health() << '\n';
    firstBot.takeDamage(25);
    std::cout << "After 25 damage: " << firstBot.health() << '\n';
    std::cout << "Alive: " << firstBot.isAlive() << '\n';

    TrainingBot secondBot{40};

    std::cout << "\nSecond bot\n";
    std::cout << "Starting health: " << secondBot.health() << '\n';
    secondBot.takeDamage(10);
    std::cout << "After 10 damage: " << secondBot.health() << '\n';
    std::cout << "Alive: " << secondBot.isAlive() << '\n';

    std::cout << "\nHeavy damage\n";
    secondBot.takeDamage(1000);
    std::cout << "Second bot health: " << secondBot.health() << '\n';
    std::cout << "Second bot alive: " << secondBot.isAlive() << '\n';

    return 0;
}
```

Final expected output

```
Training bot lab

First bot
Starting health: 100
After 25 damage: 75
Alive: true

Second bot
Starting health: 40
After 10 damage: 30
Alive: true

Heavy damage
Second bot health: 0
Second bot alive: false
```

Core lab complete. If your final program builds without relevant warnings and the output matches the example above, you have completed the required practical work.

If something is not working

Do not rewrite everything. Match the symptom you see to the first thing worth checking.

Symptom	First thing to check
TrainingBot is undefined in Main.cpp	Did you include "TrainingBot.h" at the top of Main.cpp?
Unresolved external symbol	Does the declaration in TrainingBot.h have a matching definition in TrainingBot.cpp? Is TrainingBot.cpp actually inside the project?
A const-related compiler error	Does the definition repeat const exactly where the declaration has it?
m_health is inaccessible	Good - Main.cpp should not change m_health directly. Use a public member function instead.
The bot goes below 0 health	Check the large-damage branch inside takeDamage() before subtraction happens.
The output is from an older version	Build the solution again before running. Do not keep running after a failed build.
Duplicate/redefinition message from the header	Check the #ifndef / #define / #endif include guard and the class-ending semicolon.

Final checklist

- ☐ The solution and project were created from scratch in Visual Studio.
- ☐ The project is using C++17, /W4, Debug and x64.
- ☐ Main.cpp, TrainingBot.h and TrainingBot.cpp are all inside the project.
- ☐ TrainingBot stores m_health as private data.
- ☐ takeDamage() changes health without allowing a negative value.
- ☐ health() and isAlive() are const member functions.
- ☐ Both the default constructor and TrainingBot(int) work.
- ☐ Two TrainingBot objects can hold different health values.
- ☐ The final console output matches the published output.

Short understanding check

You should now be able to answer these four questions in plain English. A one-sentence answer to each is enough.

Question	What your answer should mention
What is the difference between TrainingBot and firstBot?	TrainingBot is the class/type; firstBot is one object/instance.
Why is m_health private?	Other code should use the class operations instead of assigning any value directly.
Why is health() const?	It reads the object without changing its ordinary state.
What does : m_health{t_health} do?	It directly initialises the member before the constructor body runs.

Optional stretch - add a reset operation

Only do this after the core lab works. This extension is intentionally small: add a `reset()` member function that restores a bot to 100 health.

Step 1 - declaration

In the public section of `TrainingBot.h`, add:

```
void reset();
```

Step 2 - definition

In `TrainingBot.cpp`, add:

```
void TrainingBot::reset()
{
    // Set m_health back to 100.
}
```

Step 3 - use it

After the heavy-damage section in `Main.cpp`, reset the second bot and print the result.

```
secondBot.reset();
std::cout << "After reset: " << secondBot.health() << '\n';
```

Expected extra output

```
After reset: 100
```

Stretch complete when. The `reset` function changes the private member through the class interface and the output becomes 100. No extra class or test framework is needed.